Yuno code benchmark v4 — seven models, execution-graded

AI Squad · Yuno
2026-09-02

> **Verdict.** On the rebuilt 48-problem suite, **Gemini 3.8 Flash passes 48/48** with no friction and the cheapest run ($0.72). **Claude Fable 5.1 passes 40/48**: it refuses 8 prompts (all optimization and algorithms) but is **100% correct on everything it attempts**, closing most of the gap left by Fable 5 (16/48, 30 refusals). Kimi K3 (47), Grok 4.5 (45) and GLM 5.2 (44) keep their July order. The Claude 5 raw-API refusal pattern is still the operational risk for autonomous code agents, but it is now 8/48 instead of 30/48.

What was tested

48 in-house HumanEval-style Python problems in six categories (algorithms, data structures, optimization, bug fixing, spec compliance, edge cases; 8 each). Each model receives the problem prompt as the only user message, no system prompt (Claude: no temperature; OpenRouter models: temperature 0). Grading is by execution: the candidate module is imported in a sandboxed child process (memory and time limits) and run against the test cases; `pass` requires every case, `tests_ratio` is the fraction passed, `checker` is 1 when the module loads and defines the entry point. Refusals (`stop_reason: refusal`) and empty answers score 0 and are reported separately so they are never confused with wrong code.

The v4 suite. Both July runners were lost, so the suite was rebuilt from the problems and canonical solutions stored in Braintrust: 461 cases (6-15 per problem), expected values generated by executing the canonical solution, and a large deterministic input per optimization problem sized so a quadratic solution times out while the canonical runs in under a second. The suite was calibrated against the stored July 25 outputs of five models: 89.6% agreement on pass (215/240). Every disagreement was reviewed: 21 go in the model's favour (July tests stricter than the prompt; the outputs are behaviourally identical to the canonical under fuzzing) and 4 are real bugs July missed. Candidates run on Python 3.12 (the prompts never pin a version; on 3.9 two correct Gemini solutions failed on `int | None` and `bisect(key=)`). The suite, harness and every output now live in the `code-benchmarks` repository.

Results

MODEL	PASS (RAW)	WHEN ATTEMPTED	EMPTY	LATENCY	OUTPUT TOKENS	COST / RUN	SOURCE
Gemini 3.8 Flash	48/48 (100%)	100%	0	21.7 s	3,965	$0.72	fresh Sep 2 · OpenRouter
Kimi K3	47/48 (98%)	98%	0	65.3 s	1,981	n/a	Jul 25 outputs re-scored

Grok 4.5	45/48 (94%)	94%	0	27.5 s	1,786	n/a	Jul 25 outputs re-scored
GLM 5.2	44/48 (92%)	98%	3 max_tokens	24.7 s	4,229	n/a	Jul 25 outputs re-scored
Claude Fable 5.1	40/48 (83%)	100%	8 refusals	17.0 s	1,263	n/a	fresh Sep 2 · Anthropic API
Claude Opus 5	35/48 (73%)	100%	13 refusals	11.5 s	918	n/a	Jul 25 outputs re-scored
Claude Fable 5	16/48 (33%)	89%	30 refusals	10.3 s	717	n/a	Jul 25 outputs re-scored

Empty = API refusal (Claude 5) or reasoning that exhausted max_tokens (GLM). "When attempted" = passes ÷ answered prompts. Latency and tokens are per-problem means on answered prompts; cost is only available through OpenRouter. The five July models are their July 25 generations re-scored under v4; Fable 5.1 and Gemini 3.8 Flash are fresh September 2 generations under the same protocol.

By category (passes out of 8)

CATEGORY	GEMINI 3.8 FLASH	KIMI K3	GROK 4.5	GLM 5.2	FABLE 5.1	OPUS 5	FABLE 5
Algorithms	8/8	8/8	7/8	5/8 (3×)	5/8 (3×)	2/8 (6×)	3/8 (4×)
Data structures	8/8	8/8	7/8	8/8	8/8	8/8	5/8 (2×)
Optimization	8/8	8/8	8/8	8/8	3/8 (5×)	3/8 (5×)	1/8 (7×)
Bug fixing	8/8	8/8	8/8	8/8	8/8	6/8 (2×)	8 refused
Spec compliance	8/8	8/8	7/8	7/8	8/8	8/8	2/8 (6×)
Edge cases	8/8	7/8	8/8	8/8	8/8	8/8	5/8 (3×)

July 25 (v3) vs September 2 (v4), same outputs

MODEL	V3 PASS	V4 PASS	Δ
Kimi K3	42/48	47/48	+5
Grok 4.5	43/48	45/48	+2
GLM 5.2	39/48	44/48	+5

Claude Opus 5	32/48	35/48	+3
Claude Fable 5	15/48	16/48	+1

v3 = the July 25 reconstructed suite (since lost). Order preserved; GLM and Opus recover cases where July tests were stricter than the prompt; Grok loses two genuine ones (see below).

Findings

- **Fable 5.1 attempts far more and never misses.** 8 refusals against Fable 5's 30 on identical prompts (5 optimization, 3 algorithms; bug fixing, spec, data structures and edge cases are 8/8), and 40/40 on what it answers, with the shortest answers of the fresh runs (1263 tokens, 17.0 s). The mechanism is unchanged — the API safety classifier declines benign code prompts with an empty body — but it fires four times less often.
- **Gemini 3.8 Flash is the first model to clear the whole suite.** 48/48 at $0.72 for the run, 21.7 s per problem, verbose (3965 output tokens, second only to GLM). No refusals, no truncation.
- **Opus 5 remains the most accurate Claude when it answers** (35/35 attempted, 100%) but still refuses 13/48; those are July 25 outputs, not re-generated here.
- **What v4 changed.** Grok 4.5 loses two real cases the July suite did not exercise: it resets a re-submitted task's version in the FIFO scheduler (served at its old position) and uses `str.format` where the prompt forbids it; Grok and Fable 5 crash with `TypeError` in the colour-graph heap on cost ties. Kimi K3's only miss is competition ranking implemented as dense ranking.
- **Refusal is the operational risk, not capability.** A refusal scores 0 like a wrong answer and is invisible unless `stop_reason` is inspected. Every Claude 5 refusal in this suite came from bare code prompts over the raw API; the Maia agent evals run on Fable without a single refusal.

Recommendation

For autonomous code agents on the raw API, Gemini 3.8 Flash, Kimi K3 and Grok 4.5 are the frictionless choices, with Gemini now leading on accuracy and cost. Fable 5.1 is viable where a supervising loop can detect a refusal and re-route (its accuracy is the best in the set), and its refusal rate should be re-measured on each model release since it moved from 30 to 8 between Fable 5 and 5.1. Keep the suite in the repository and re-run `calibrate` before every comparison.

Appendix — non-refusal failures per model

MODEL	CATEGORY	PROBLEM	FIRST FAILING CASE
Kimi K3	edge_cases	`competition-ranks-with-ties-and-nan-guard`	ties_skip_ranks: expected {"value": [3, 1, 1, 4]} got {"ok": true, "loaded": true, "value": [2, 1, 1, 3]}
Grok 4.5	data_structures	`reprioritizable-task-scheduler-with-fifo-ties`	cancel_then_resubmit_gets_fresh_fifo: expected {"value": [2, 3, "a", "c", "b", 0]} got {"ok": true, "loaded": true, "value": [2, 3, "a", "b", "c", 0]}
Grok 4.5	spec_compliance	`placeholder-template-renderer-with-exact-error-positions`	no_import_no_format: expected {"value": [false, [], []]} got {"ok": true, "loaded": true, "value": [false, ["format"], []]}

Grok 4.5	algorithms	`cheapest-walk-with-color-switch-surcharge`	large_random_graph: expected {"value": 47} got {"ok": false, "loaded": true, "exc": "TypeError", "exc_msg": "'<' not supported between instances of 'N
GLM 5.2	spec_compliance	`duration-string-parser-under-a-no-import-contract`	ascii_digits_only: expected {"value": [["ValueError", true], ["ValueError", true], ["ValueError", true], ["ValueError", true], ["ValueError", true]] g
Claude Fable 5	data_structures	`reprioritizable-task-scheduler-with-fifo-ties`	import: SyntaxError: invalid syntax (sol.py, line 1)
Claude Fable 5	algorithms	`cheapest-walk-with-color-switch-surcharge`	large_random_graph: expected {"value": 47} got {"ok": false, "loaded": true, "exc": "TypeError", "exc_msg": "'<' not supported between instances of 's

Sources: Braintrust project "code-benchmarks", experiments code-bench-v4__* (7 models × 48 rows, per-test detail in metadata); repository code-benchmarks (harness, tests, outputs, this report). Prepared by Andrés Espitia with Claude Code.